Smart Pilgrim Queue Management System (SPQMS)

Software Engineering Project Documentation

---

1. Project Overview

The **Smart Pilgrim Queue Management System (SPQMS)** is a web-based Software Engineering project developed to digitize and manage pilgrim darshan bookings, live queue tracking, daily SSD (Sarva Darshan Sarva Token) quota information, temporary QR-based exit/re-entry passes, and administrative temple queue controls for Tirumala temple operations.

The system provides a centralized digital workflow that connects three primary user roles—**Pilgrims (Users)**, **Temple Administrators**, and **Temple Verification Staff**—to ensure smooth crowd management, queue visibility, and operational transparency without physical queue congestion.

---

2. Problem Statement

Pilgrims visiting Tirumala face significant physical challenges in traditional queue management systems:

- **Lack of Real-Time Queue Visibility:** Pilgrims waiting in physical compartments (*compartments/vaikuntam*) often have no clear information on their exact position or expected waiting time.
- **Physical Exit and Re-entry Friction:** Pilgrims needing temporary exits for medical reasons, restrooms, or food risk losing their queue slot due to lack of verified exit documentation.
- **Daily SSD Token Dissemination:** Difficulty in publicly monitoring the daily release, utilization, and remaining quota of free Sarva Darshan (SSD) tokens.
- **Manual Administrative Overhead:** Temple authorities lack a unified digital portal to configure real-time queue counters, adjust processing rates, issue daily quotas, and verify exit passes.

---

3. Proposed Solution

SPQMS provides a modular, lightweight web portal that addresses these operational challenges using deterministic software engineering logic:

1. **Digital Darshan Booking:** Online booking portal for bookable darshan categories (e.g., Special Entry Darshan ■300) supporting single and multi-pilgrim detail entry with government ID verification.

2. **Real-Time Queue Tracking:** Deterministic queue position calculation, people-ahead count, formatted waiting time estimation (e.g., `2 hrs 24 mins`), and congestion level indicators (`LOW`, `MEDIUM`, `HIGH`).

3. **Daily SSD Quota Visibility:** Real-time visibility into the daily SSD quota released by temple authorities, tokens used, and remaining quota for walk-in pilgrims.

4. **Temporary QR Exit Pass Generation & Verification:** Self-service QR exit pass generation for active queue pilgrims, paired with a dedicated scanner verification portal for security staff at exit/entry gates.

5. **Role-Based Portals:** Separate, secure interfaces for Pilgrims, Admin, and Temple Staff.

---

4. Objectives

- Digitize darshan information, booking, and queue management workflows.

- Provide pilgrims with accurate queue position and deterministic waiting-time visibility.
- Eliminate raw mathematical formulas and technical clutter from user-facing screens.
- Implement a tamper-proof QR exit pass verification process for temple gate staff.
- Centralize daily SSD token quota release and tracking.
- Deliver a clean, professional, human-designed user interface adhering to modern accessibility and Software Engineering standards.

---

5. Scope of the System

In-Scope (Implemented SWE Features):

- Pilgrim user registration, secure session authentication, and profile management.
- Public darshan category browsing with clear separation of online bookable vs. walk-in types.
- Multi-pilgrim booking flow with instant ticket confirmation and unique booking IDs.
- Deterministic queue algorithm calculating live position, people ahead, and formatted wait time.
- Daily SSD quota monitoring widget and admin release workflow.
- QR exit pass creation (max 120 minutes validity) and temple staff scanning verification.
- Admin dashboard displaying live metrics, queue configuration, and booking lists.
- User notification system for booking updates and pass alerts.

Out-of-Scope (Not Included / Separate):

- Machine Learning crowd prediction or AI-based waiting time estimation models.
- External payment gateway integration (simulated payment flow is implemented).
- Direct SMS gateway integration (in-app notifications are implemented).

---

6. System Users / Roles

The system defines three distinct user roles with specific access boundaries:

User Role	Access Boundary	Key Responsibilities
Pilgrim / User	User Portal (`/dashboard`)	Register/Login, view darshan types, book slots, check queue status, generate QR exit passes, view SSD quota, view notifications.
Admin	Admin Portal (`/admin/dashboard`)	View system metrics, release daily SSD quota, update queue counter parameters (processing rate, active counters), monitor bookings and QR passes.
Temple Staff	Staff Portal (`/staff/dashboard`)	Scan or manually verify pilgrim QR exit passes at gate locations, record exit and re-entry events in real time.

---

7. Functional Modules

```
SPQMS Modular Structure
 ■■■ User Management Module (Implemented)
 ■■■ Darshan Management Module (Implemented)
 ■■■ Booking Management Module (Implemented)
 ■■■ Queue Management Module (Implemented)
 ■■■ SSD Quota Management Module (Implemented)
 ■■■ QR Temporary Exit Module (Implemented)
 ■■■ Admin Management Module (Implemented - 50% Core Operational Scope)
 ■■■ Temple Staff Portal Module (Implemented)
 ■■■ Notification Module (Implemented)
```

---

8. System Architecture

SPQMS follows a standard 3-tier RESTful Web Architecture:

```
+-----------------------------------------------------+
|                 React Frontend (SPA)                |
|   (User Portal | Admin Portal | Staff Verification)   |
+--------------------------+--------------------------+
                           |
                           | HTTP / REST API (JSON)
                           v
+-----------------------------------------------------+
|               FastAPI Backend (Python)              |
|  (Auth, Booking Engine, Queue Calculator, QR Verification) |
+--------------------------+--------------------------+
                           |
                           | SQL Queries (sqlite3)
                           v
+-----------------------------------------------------+
|              SQLite Database (Local DB)             |
| (USERS, SESSIONS, DARSHAN, BOOKINGS, QUEUE, QR, SSD) |
+-----------------------------------------------------+
```

---

9. Technology Stack

Technology	Role / Purpose	Details / Version
React	Frontend Framework	Single Page Application (SPA), React Router v6
JavaScript (ES6+)	Frontend Logic	State management, Fetch API
CSS3	User Interface	Custom responsive design system (Burgundy & Soft Off-White)
Python 3.10+	Backend Language	Fast asynchronous REST backend
FastAPI	Web Framework	API routing, request validation, Pydantic schemas
SQLite 3	Database	Embedded relational database engine (`smart_darshan.db`)
Uvicorn	ASGI Server	High-performance asynchronous server implementation
PBKDF2-HMAC-SHA256	Password Hashing	100,000 iterations with 16-byte random salt

Session Tokens	Authentication	64-character hex security token with expiration
QRCode / Canvas	QR Code Generation	Client-side & server-side string encoding

---

10. Database Design

The relational schema is stored in `backend/smart_darshan.db` and initialized programmatically via `backend/database.py`. Relational integrity is enforced using foreign keys across primary tables.

---

11. Database Tables

1. `USERS` Table

Stores registered accounts for pilgrims, administrators, and verification staff.

- `id` (INTEGER, Primary Key, Auto Increment)
- `email` (TEXT, Unique, Not Null)
- `password_hash` (TEXT, Not Null)
- `salt` (TEXT, Not Null)
- `full_name` (TEXT, Not Null)
- `phone` (TEXT)
- `role` (TEXT, Default 'User') — Options: 'User', 'Admin', 'Staff'
- `created_at` (TIMESTAMP)

2. `ADMINS` Table

Links admin users to department metadata.

- `id` (INTEGER, Primary Key)
- `user_id` (INTEGER, Foreign Key -> `USERS.id`)
- `department` (TEXT)
- `access_level` (TEXT)

3. `STAFF` Table

Links staff accounts to physical gate locations.

- `id` (INTEGER, Primary Key)
- `user_id` (INTEGER, Foreign Key -> `USERS.id`)
- `gate_location` (TEXT)
- `employee_id` (TEXT)

4. `SESSIONS` Table

Manages active authentication session tokens.

- `token` (TEXT, Primary Key)
- `user_id` (INTEGER, Foreign Key -> `USERS.id`)

- `created_at` (TIMESTAMP)
- `expires_at` (TIMESTAMP)

5. `DARSHAN` Table

Defines available darshan service categories.

- `id` (INTEGER, Primary Key)
- `name` (TEXT, Not Null)
- `description` (TEXT)
- `price` (REAL, Default 0.0)
- `available_slots` (INTEGER, Default 0)
- `is_bookable` (INTEGER, Default 1) — `0` for Walk-in (Sarvadarsanam), `1` for Online Bookable.

6. `BOOKINGS` Table

Stores main booking records.

- `id` (INTEGER, Primary Key)
- `booking_id` (TEXT, Unique, Not Null) — e.g. `SPQ-104928`
- `user_id` (INTEGER, Foreign Key -> `USERS.id`)
- `darshan_id` (INTEGER, Foreign Key -> `DARSHAN.id`)
- `booking_date` (TEXT, Not Null)
- `time_slot` (TEXT, Not Null)
- `num_pilgrims` (INTEGER, Default 1)
- `status` (TEXT, Default 'CONFIRMED')
- `created_at` (TIMESTAMP)

7. `BOOKING_PILGRIMS` Table

Stores details of each pilgrim included in a booking.

- `id` (INTEGER, Primary Key)
- `booking_id` (TEXT, Foreign Key -> `BOOKINGS.booking_id`)
- `name` (TEXT, Not Null)
- `age` (INTEGER, Not Null)
- `gender` (TEXT, Not Null)
- `id_type` (TEXT)
- `id_number` (TEXT)

8. `QUEUE` Table

Tracks active queue position and entry timestamp.

- `id` (INTEGER, Primary Key)
- `booking_id` (TEXT, Foreign Key -> `BOOKINGS.booking_id`)
- `user_id` (INTEGER, Foreign Key -> `USERS.id`)
- `queue_number` (INTEGER, Not Null)
- `status` (TEXT, Default 'WAITING')
- `entry_time` (TIMESTAMP)

9. `QUEUE_SETTINGS` Table

Stores operational configuration for wait-time calculations.

- `id` (INTEGER, Primary Key)
- `avg_processing_mins` (REAL, Default 3.0)
- `active_counters` (INTEGER, Default 5)
- `buffer_factor` (REAL, Default 1.1)
- `updated_at` (TIMESTAMP)

10. `SSD_QUOTA` Table

Manages daily Sarva Darshan token quota records.

- `id` (INTEGER, Primary Key)
- `quota_date` (TEXT, Unique, Not Null)
- `total_quota` (INTEGER, Not Null)
- `tokens_used` (INTEGER, Default 0)
- `is_active` (INTEGER, Default 1)

11. `QR_EXIT_PASS` Table

Tracks temporary exit passes.

- `id` (INTEGER, Primary Key)
- `pass_code` (TEXT, Unique, Not Null) — e.g. `PASS-849201`
- `user_id` (INTEGER, Foreign Key -> `USERS.id`)
- `booking_id` (TEXT, Foreign Key -> `BOOKINGS.booking_id`)
- `reason` (TEXT)
- `exit_time` (TIMESTAMP)
- `return_deadline` (TIMESTAMP)
- `status` (TEXT, Default 'ACTIVE') — Options: 'ACTIVE', 'USED', 'EXPIRED'

12. `NOTIFICATIONS` Table

Stores in-app messages for pilgrims.

- `id` (INTEGER, Primary Key)
- `user_id` (INTEGER, Foreign Key -> `USERS.id`)
- `title` (TEXT, Not Null)
- `message` (TEXT, Not Null)
- `is_read` (INTEGER, Default 0)
- `created_at` (TIMESTAMP)

---

12. Frontend Implementation

The frontend is built as a single-page React application structured into modular components and pages.

Page / Component	Route	Description	Implementation Status

Home	`/`	Dashboard-style introduction with compact overview cards.	Completed
Login	`/login`	Pilgrim authentication page with error alerts.	Completed
Register	`/register`	Pilgrim account creation page.	Completed
User Dashboard	`/dashboard`	Central user portal showing profile, active booking, SSD quota widget, notifications.	Completed
Darshan Types	`/darshan-types`	Card layout showing bookable and walk-in darshan packages.	Completed
Book Darshan	`/book-darshan`	Multi-step form with slot selection, multi-pilgrim details, summary, and payment simulation.	Completed
Queue Status	`/queue-status`	Clean card interface displaying position, people ahead, formatted wait time (`2 hrs 24 mins`), and congestion badge.	Completed
SSD Quota	`/ssd`	Public daily SSD token quota monitor (Released, Used, Remaining).	Completed
QR Exit Pass	`/qr-pass`	Pass request form and active exit pass ticket with QR code display.	Completed
Notifications	`/notifications`	In-app notification list with read status toggles.	Completed
Admin Login	`/admin/login`	Portal login for temple administrative staff.	Completed
Admin Dashboard	`/admin/dashboard`	Administrative management panel for queue settings, SSD quota release, and booking overview.	Completed (50% Core Scope)
Staff Login	`/staff/login`	Security staff authentication.	Completed
Staff Verification	`/staff/dashboard`	QR scanner/manual code verifier portal for gate personnel.	Completed

---

13. Backend Implementation

The backend is built with **FastAPI** (`backend/main.py`), utilizing Pydantic models for strict data validation and `sqlite3` for fast, lightweight data access.

Key backend services:

- **Authentication Engine:** PBKDF2 password hashing with custom random salt generation and bearer session token verification middleware.

- **Booking Engine:** Generates unique `SPQ-` prefixed IDs, creates pilgrim entries in `BOOKING_PILGRIMS`, auto-assigns queue positions, and sends confirmation notifications.
- **Deterministic Queue Engine:** Calculates live position and expected wait times based on operational counter parameters.
- **QR Pass Manager:** Issues unique `PASS-` codes, enforces 2-hour maximum return windows, and validates pass status upon scanning.

---

14. API Endpoints

Method	Endpoint	Description	Auth Required	Status
`GET`	`/`	System root health check	No	Completed
`POST`	`/register`	Register new user account	No	Completed
`POST`	`/login`	Authenticate User, Admin, or Staff	No	Completed
`GET`	`/auth/me`	Fetch active user session profile	Yes	Completed
`POST`	`/logout`	Invalidate active session token	Yes	Completed
`GET`	`/darshan`	List all darshan categories	No	Completed
`GET`	`/darshan/bookable`	List online bookable darshan types	No	Completed
`POST`	`/booking`	Create darshan booking (single/multi pilgrim)	Yes	Completed
`GET`	`/bookings`	List user's booking history	Yes	Completed
`GET`	`/booking/{booking_id}`	Get specific booking details	Yes	Completed
`GET`	`/queue/{booking_id}`	Fetch live queue status & wait time	Yes	Completed
`POST`	`/qr/generate`	Request temporary exit pass	Yes	Completed
`GET`	`/qr/{qr_id}`	Fetch exit pass details	Yes	Completed
`GET`	`/qr/booking/{id}`	Get active exit pass for booking	Yes	Completed
`POST`	`/qr/verify`	Staff QR pass scanner verification	Yes (Staff)	Completed
`GET`	`/ssd/daily`	Fetch daily SSD quota summary	No	Completed
`POST`	`/admin/ssd/release`	Admin release/update daily SSD quota	Yes (Admin)	Completed

POST	/admin/queue-settings	Admin update counter operational parameters	Yes (Admin)	Completed
GET	/admin/dashboard-stats	Fetch aggregate admin dashboard metrics	Yes (Admin)	Completed
GET	/notifications	List user notifications	Yes	Completed
POST	/notifications/{id}/read	Mark notification as read	Yes	Completed

---

15. Authentication Flow

```
User Registration:
[Enter Email, Password, Name] ■■> [Generate Salt] ■■> [PBKDF2 Hash] ■■> [Store in USERS]

User Login:
[Enter Credentials] ■■> [Fetch Salt & Hash] ■■> [Verify PBKDF2] ■■> [Generate UUID Session Token] ■■> [Stor
e in SESSIONS & Return Token]

Authenticated Request:
[Request with Header "Authorization: Bearer <token>"] ■■> [Check SESSIONS] ■■> [Return User Profile]
```

---

16. Darshan Booking Flow

1. Pilgrim selects an online bookable darshan (e.g., Special Entry Darshan ■300).
2. Sarva Darshan (Free Walk-in) is displayed as an informational category with a walk-in badge and is excluded from online booking.
3. Pilgrim selects date and available time slot (e.g., `08:00 AM - 12:00 PM`).
4. Pilgrim adds details for all attending pilgrims (Name, Age, Gender, ID Type, ID Number).
5. User reviews summary and completes simulated payment.
6. Backend creates booking record (`SPQ-XXXXXX`), inserts pilgrim entries, assigns queue position (`#QUEUE`), and creates a confirmation notification.

---

17. Queue Management Flow

The queue module relies on deterministic software logic calculated dynamically in the backend:

People Ahead = \max(0, Queue Number - 1)

Processing Rate (persons/min) = \frac{Active Counters{Avg Processing Mins per Person

Estimated Minutes = \lceil (People Ahead \times Avg Processing Mins / Active Counters) \times Buffer Factor \rceil

Presentation Rule:

Mathematical formulas are strictly executed on the backend. The user interface presents formatted results only:

- **Queue Position:** `#43`
- **People Ahead:** `42`
- **Estimated Waiting Time:** `2 hrs 24 mins` (or `144 mins`)

- **Queue Status:** `HIGH` / `MEDIUM` / `LOW` badge

---

18. QR Temporary Exit Flow

1. Pilgrim currently waiting in the queue requests a temporary exit pass from `/qr-pass`.

2. Selects reason (e.g., Restroom, Medical, Food/Water) and expected return time.

3. Backend issues a unique pass (`PASS-XXXXXX`), sets `exit_time = NOW()`, and sets `return_deadline = NOW() + 120 mins`.

4. The pass generates a scannable QR Code on the pilgrim's screen.

5. Temple staff at the exit gate opens the **Staff Portal** (`/staff/dashboard`) and scans or enters the pass code.

6. Backend endpoint `/qr/verify` validates the pass status (`ACTIVE` vs. `EXPIRED/USED`), records verification, and updates status.

---

19. SSD Quota Management

SSD (Sarva Darshan Sarva Token) is implemented strictly as a **Daily Temple Quota**.

- It represents the total free walk-in tokens released by temple management for a specific calendar date based on crowd expectations.
- Admin releases or updates the quota via `POST /admin/ssd/release`.
- Pilgrims view aggregate daily quota metrics:
- **Tokens Released:** `5,000`
- **Tokens Used:** `3,240`
- **Tokens Remaining:** `1,760`
- **Status:** `AVAILABLE`

---

20. Admin Management

The Admin Portal (`/admin/dashboard`) provides operational controls:

- **Metric Cards:** Total Bookings, Active Queue Count, Daily SSD Quota Overview, Current Congestion Status.
- **SSD Quota Release Panel:** Set calendar date, release token count, update active status.
- **Queue Settings Panel:** Adjust active counters (e.g., 5 to 10 counters) and average processing time per pilgrim (e.g., 2.5 mins).
- **Bookings & Exit Pass Tables:** Real-time table viewing recent pilgrim bookings and active QR exit passes.

---

21. Temple Staff Verification Portal

The Staff Portal (`/staff/dashboard`) is a dedicated role interface for security personnel:

- Input/scanner field for entering `PASS-XXXXXX` pass codes.
- Instant validation display indicating:
- **VALID / APPROVED (Green):** Pass active, deadline unexpired.
- **EXPIRED / INVALID (Red):** Deadline exceeded or fake pass code.

• Displays pilgrim name, booking reference, exit timestamp, and return deadline.

---

22. Notifications

In-app notifications inform pilgrims of key events:

• Booking Confirmation (`Booking SPQ-XXXXXX Confirmed`).

• QR Exit Pass Generation (`Exit Pass PASS-XXXXXX Created`).

• Queue status updates.

Notifications are accessible via `/notifications` and the user dashboard header.

---

23. User Interface Screens

Screen Name	Location	Visual Theme & Key Elements
Home Page	`/`	Clean off-white background, compact dashboard introduction, 4 service overview cards.
User Dashboard	`/dashboard`	Pilgrim profile header, active queue summary card, SSD quota widget, quick action links.
Darshan Types	`/darshan-types`	Grid of service cards with pricing, descriptions, andWalk-in/Book Now badges.
Darshan Booking	`/book-darshan`	Step-by-step clean wizard for slot selection, multi-pilgrim detail entry, and confirmation.
Queue Status	`/queue-status`	Focus card showing position #43, people ahead 42, wait time `2 hrs 24 mins`, congestion status.
SSD Quota View	`/ssd`	Clean summary card with Released, Used, Remaining, and Progress indicator.
QR Exit Pass	`/qr-pass`	Simple request form & QR pass display card with return deadline timer.
Notifications	`/notifications`	List of system messages with unread/read badges.
Admin Dashboard	`/admin/dashboard`	Professional management portal with metric cards, control forms, and live monitoring tables.
Staff Verification	`/staff/dashboard`	Clean scanner interface for gate staff with pass verification feedback banners.

---

24. Current Implementation Status

Module	Implementation Status	Remarks
User Management	**Completed**	Registration, Login, Session Auth, PBKDF2 Password Hashing.
Darshan Management	**Completed**	Listing, walk-in tagging, bookable slot filtering.
Booking Management	**Completed**	Multi-pilgrim details, slot allocation, unique ID generation.
Queue Management	**Completed**	Dynamic queue numbers, position tracking, wait time estimation.
SSD Quota Management	**Completed**	Daily quota summary view, admin release endpoint.
QR Temporary Exit	**Completed**	QR generation, 120-min deadline enforcement, staff scanner verification.
Admin Management	**Completed (50% Core Scope)**	Metrics dashboard, SSD release form, counter settings, live tables.
Temple Staff Portal	**Completed**	Dedicated portal, pass verification logic, pass status updates.
Notification System	**Completed**	User notifications table, read/unread states, automatic event triggers.

---

25. Features Completed

- Full authentication system with session token invalidation on logout.
- Multi-pilgrim darshan booking with pilgrim name, age, gender, and ID verification.
- Deterministic queue calculation engine.
- Formatted human-readable wait time UI without technical formulas.
- Scannable QR exit pass workflow with staff scanner portal.
- Admin controls for real-time queue counter adjustments.
- Daily SSD quota release and usage monitoring.
- Unified burgundy/soft off-white design system across all screens.

---

26. Features Partially Completed

- **Admin Management (50% Target Scope):** Core administrative queue adjustments, SSD releases, and metric tables are complete; advanced historical analytics and multi-gate audit reporting are reserved for future phases.

---

27. Features Pending

- Physical hardware barcode/QR camera scanner integration (currently uses scanner code input / browser string verification).
- Automated SMS gateway integration (currently uses in-app notification system).

- Live payment gateway integration (currently uses simulated instant payment approval).

---

28. Testing Performed

Functional testing was conducted via direct API unit scripts (`scratch/test_endpoints.py`) and browser verification:

- **Auth Testing:** Verified password salt generation, hash verification, incorrect password rejection, and session token expiry.
- **Booking Flow Testing:** Verified multi-pilgrim detail persistence, queue slot auto-increment, and booking lookup by ID.
- **Queue Engine Testing:** Tested position updates when queue parameters (counters, processing time) are modified by admin.
- **QR Exit Pass Testing:** Verified pass generation, valid verification response, return deadline check, and double-scan rejection (`ALREADY_USED`).
- **SSD Quota Testing:** Verified quota calculation (`Total - Used = Remaining`) and daily date lookup.

Note: Functional testing was performed directly via Python test suites and browser manual testing.

---

29. Known Issues / Limitations

- Database runs on SQLite, which is ideal for single-instance local deployments but requires PostgreSQL migration for high-concurrency production deployments.
- QR scanner relies on text/barcode string input or simulated camera scanning in browser.

---

30. Project Folder Structure

```
Tirumala-Smart-Darshan/
■■■ backend/
■   ■■■ main.py              # FastAPI REST endpoints & application entry
■   ■■■ database.py          # SQLite schema initialization & database connection
■   ■■■ schemas.py           # Pydantic data schemas & request validation
■   ■■■ auth_utils.py        # PBKDF2 password hashing & session management
■   ■■■ smart_darshan.db     # SQLite database file
■
■■■ frontend/
■   ■■■ public/              # Static HTML shell & assets
■   ■■■ src/
■   ■   ■■■ components/      # Reusable UI components
■   ■   ■■■ pages/           # Portal page components
■   ■   ■   ■■■ Home.js
■   ■   ■   ■■■ Login.js / Register.js
■   ■   ■   ■■■ UserDashboard.js
■   ■   ■   ■■■ DarshanTypes.js
■   ■   ■   ■■■ BookDarshan.js
■   ■   ■   ■■■ QueueStatus.js
■   ■   ■   ■■■ SSDQuotaView.js
■   ■   ■   ■■■ QRExitPass.js
■   ■   ■   ■■■ Notifications.js
■   ■   ■   ■■■ AdminDashboard.js / AdminLogin.js
■   ■   ■   ■■■ StaffDashboard.js / StaffLogin.js
■   ■   ■■■ App.js           # Main routing & layout controller
■   ■   ■■■ App.css          # Design system CSS (Burgundy & Soft Off-white)
■   ■   ■■■ index.js         # React DOM renderer
■   ■■■ package.json         # Node.js dependencies & scripts
■
■■■ SPQMS_PROJECT_DOCUMENTATION.md  # Comprehensive SWE Documentation
■■■ README.md                      # Repository overview & setup instructions
```

---

31. How to Run the Project

Prerequisites:

- Python 3.10 or higher
- Node.js v16 or higher & npm

Step 1: Start Backend Server

```
# Navigate to project root directory
cd Tirumala-Smart-Darshan

# Install backend dependencies (if needed)
pip install fastapi uvicorn pydantic passlib

# Launch FastAPI server on port 8000
python -m uvicorn backend.main:app --reload --port 8000
```

Backend runs at: `http://localhost:8000`

Interactive API Docs at: `http://localhost:8000/docs`

Step 2: Start Frontend Application

```
# Open a new terminal and navigate to frontend
cd Tirumala-Smart-Darshan/frontend

# Install Node modules (if needed)
npm install

# Start React development server
npm start
```

Frontend runs at: `http://localhost:3000`

Demo Portals & Credentials:

- **User Portal:** `http://localhost:3000/login` (`pilgrim@example.com` / `user123`)
- **Admin Portal:** `http://localhost:3000/admin/login` (`admin@tirumala.org` / `admin123`)
- **Staff Portal:** `http://localhost:3000/staff/login` (`staff@tirumala.org` / `staff123`)

---

32. GitHub Repository

- **Repository URL:**
  https://github.com/Nikhil64-stack/Tirumala-Smart-Darshan

---

33. Future Enhancements

- Integration with production-grade PostgreSQL / MySQL database.
- Native mobile app wrapper (React Native) for pilgrim QR pass display.
- Physical QR scanner hardware integration at gate kiosks.
- SMS gateway integration (Twilio / Fast2SMS) for queue alerts.
- Multi-language localization (Telugu, Tamil, Kannada, Hindi, English).

---

34. Conclusion

The **Smart Pilgrim Queue Management System (SPQMS)** successfully provides a practical, robust, and clean Software Engineering solution for Tirumala pilgrim queue tracking, booking management, daily SSD quota visibility, and QR-based gate verification. By utilizing deterministic backend software logic and removing unnecessary technical formulas and marketing fluff from user screens, SPQMS delivers a realistic, professional, and accessible web application tailored for real-world temple queue management.